

Pearls AQI Predictor

Interview Preparation Guide

Hasnain Irshad | 10Pearls Data Science Internship

Reading note: This PDF is a print-friendly copy of the complete Markdown interview guide. The Markdown file remains the editable source.

```
# Pearls AQI Predictor: Interview Preparation Guide
```

```
**Candidate:** Hasnain Irshad
**Internship:** 10Pearls Data Science Internship
**Repository:** https://github.com/Hasnain-irshad/10pearls-aqi-predictor
**Live dashboard:** https://www.10pearlsaqi.me
**Backend API:** https://aqi-backend-production-5af4.up.railway.app
**Scope:** 22 Pakistani cities, 72-hour AQI forecasting, automated MLOps
```

This guide is a speaking document, not a script to memorize word for word. Use the short answers first, then expand with the technical detail when the interviewer asks.

1. The 30-Second Answer

```
> I built Pearls AQI Predictor, an end-to-end machine-learning system that forecasts
> the US EPA Air Quality Index up to 72 hours ahead for 22 cities across Pakistan.
> It fetches weather and pollution data from Open-Meteo, engineers leakage-controlled
> features, stores them through a Hopworks-or-Parquet storage layer, trains and gates
> candidate models, and publishes forecasts through a FastAPI backend and React/Vite
> dashboard. XGBoost is the production champion with validation RMSE 19.69, MAE 12.80,
> and R-squared 0.85. The system also includes prediction intervals, global and local
> SHAP explanations, statistical analytics, model evaluation, drift monitoring, a
> What-If simulator, and a grounded MCP/LLM advisor. GitHub Actions runs the hourly
> feature and forecast publication workflow and the daily training workflow.
```

One-minute version

The original problem was that air-quality information is usually a current reading, not an actionable forecast. I designed a global multi-horizon model: the forecast horizon is an input feature, so one model can serve ten lead times from 1 to 72 hours and all 22 cities. The pipeline uses actual historical weather during training and forecast weather during inference, with recent pollution state as anchor features.

The engineering challenge was making the model a product rather than a notebook. Feature ingestion, inference, training, evaluation, monitoring and deployment are separate concerns. GitHub Actions provides scheduled compute, Hopworks provides the feature store and model registry when credentials are available, committed JSON snapshots provide the serving contract, Railway hosts FastAPI, and Vercel hosts the React dashboard. A champion-challenger gate prevents an equal or worse model from replacing the current champion.

2. Project Facts to Memorize

Area	Final project fact
Target	Open-Meteo `us_aqi`, the US EPA AQI scale
Coverage	22 cities across Pakistan
Forecast	Hourly values for the next 72 hours plus a 3-day aggregation
Raw data	Weather and pollutant observations from Open-Meteo
Features	74 engineered columns; 27 model input columns
Champion	Global XGBoost regressor
Validation	Chronological split: 160,002 train and 39,998 validation rows
Champion metrics	RMSE 19.69, MAE 12.80, R-squared 0.850

```
| Walk-forward | Mean RMSE 20.55 with standard deviation 3.53 across 5 folds |
| Intervals | Empirical 10th/90th residual quantiles, an 80% horizon-aware interval |
| Dashboard | React/Vite, Recharts, react-leaflet, custom Aurora design |
| Backend | FastAPI/Uvicorn on Railway |
| Frontend | Vercel at `www.10pearlsaqi.me` |
| Automation | GitHub Actions: hourly feature/inference, daily training/evaluation |
| Tests | 33 passing tests |
| License | MIT |
```

3. Problem and Impact

What problem does the project solve?

People need to make decisions before pollution arrives: outdoor work, school travel, exercise, masks and precautions for vulnerable people. A current AQI reading does not provide that lead time. The project turns weather, recent pollution and calendar signals into a 72-hour city-level forecast.

Why Pakistan and why multiple cities?

Pakistan has strong seasonal and geographic variation in air quality. A Lahore-only model would miss industrial cities, coastal cities, northern cities and different weather regimes. A global model over 22 cities gives broader coverage and lets the model learn location effects through latitude, longitude and shared weather/pollution relationships.

What is the target exactly?

The model predicts Open-Meteo's `us\_aqi` value. That value is an AQI index, not a pollutant concentration. The project also contains an AQI computation module for categories, education and fallback labeling, but the forecast target uses the provider's supplied `us\_aqi` because it includes the provider's averaging logic.

What are the limitations?

- The target is a modeled atmospheric product, not a reference-grade station reading.
- Training uses observed future weather while inference uses weather forecasts, so operational error can be higher than validation error.
- The interval is calibrated from residual quantiles and still needs prospective coverage measurement.
- The system is city-level and hourly; it is not street-level or sub-hourly.
- Gas-pollutant AQI conversion needs careful unit conversion and is documented as future work; trusted particulate inputs are used for local fallback computation.

4. Architecture

```
```text
Open-Meteo
 |
 v
Hourly feature pipeline: fetch -> engineer -> Hopsworks or local Parquet
 | |
 | v
 | Daily training pipeline
 | |
 | candidates -> gate -> registry
 | |
 | v
Hourly inference pipeline <----- champion model
 |
 +--> predictions.json, intervals, alerts and SHAP explanations
 |
 +--> GitHub commit -> Railway FastAPI -> Vercel React dashboard
```

Analytics, evaluation, monitoring and SHAP snapshots are also committed JSON

```
artifacts. The MCP/LLM advisor calls the same grounded forecast tools.
'''
```

### ### The main components

1. **Open-Meteo client**: fetches air quality and weather with retries and explicit `Asia/Karachi` timezone handling.
2. **Feature pipeline**: fetches seven days of context, builds features for all cities, and upserts on `(city, timestamp)`.
3. **Backfill**: one-off historical loading in chunks, with history-aware resume.
4. **Training pipeline**: reads features, builds supervised data, compares models, computes intervals and applies the promotion gate.
5. **Inference pipeline**: loads the champion, fetches current conditions and future weather, predicts all cities, and writes the forecast document.
6. **Published artifact layer**: Railway reads the latest repository JSON at runtime with a short cache and bundled fallback. Vercel bundles the same snapshots for static fallback.
7. **FastAPI**: serves forecasts, evaluation, monitoring, SHAP, What-If and advisor endpoints.
8. **React dashboard**: Forecast, Analytics & SHAP, Model Evaluation, Monitoring and What-If views.

### ### Why separate pipelines?

Separation gives each job a clear responsibility and failure boundary. Feature refresh can run hourly without retraining. Training can fail without deleting the last champion. Inference can publish a fresh forecast using the existing champion. The storage and artifact boundaries make the system reproducible and easier to debug.

## ## 5. Technology Choices

Choice	Why it fits	Trade-off
Open-Meteo	Free, keyless, weather and air-quality history/forecast	Provider target is modeled rather than station measured
Python 3.11	Strong compatibility with pandas, scikit-learn, XGBoost and deployment	Older than the newest Python releases
Hopsworks	Feature store and model registry with versioning	Free-tier jobs can be slow; local Parquet fallback is required
GitHub Actions	Scheduled, visible, repository-native compute	Runtime limits and dependency-install time
XGBoost	Strong performance on structured tabular features	Less naturally sequential than an LSTM
FastAPI	Typed, fast API with automatic OpenAPI docs	Separate frontend/backend deployment requires CORS and deployment configuration
React/Vite	Better dashboard interaction and layout control than a quick demo framework	More frontend code than Streamlit
Recharts	Responsive model and EDA charts	Bundle size is larger; code splitting could improve it
SHAP TreeExplainer	Exact, useful tree-model attributions	Explanation computation adds runtime/dependency cost
Railway + Vercel	Simple Docker backend and static frontend deployment	Artifact publication must be automated to avoid stale data

## ## 6. Data and Feature Engineering

### ### Raw variables

The ingestion layer combines pollutant variables such as PM2.5, PM10, carbon monoxide, nitrogen dioxide, sulphur dioxide, ozone and AQI with weather variables including temperature, humidity, dew point, precipitation, pressure, cloud cover, wind speed, wind direction and gusts.

### ### Feature families

- **Calendar**: hour, day of week, month, day of year and weekend indicator.
- **Cyclical**: sine/cosine encodings for hour, month and day-of-week.
- **Wind vectors**: east-west and north-south components derived from speed/direction.
- **Lags**: AQI, PM2.5 and PM10 at 1, 3, 6, 12 and 24 hours.
- **Rolling statistics**: 6-hour and 24-hour means, standard deviations and maxima.
- **Change features**: AQI differences and relative change.
- **Location**: latitude, longitude and city context.
- **Forecast horizon**: the number of hours ahead being predicted.

### Why use cyclical encoding?

A raw hour treats 23 and 0 as far apart numerically, even though they are adjacent. Sine/cosine maps them onto a circle, preserving periodic structure. The same logic applies to month and day-of-week.

### How was leakage prevented?

Every rolling feature is shifted before aggregation, so the current target row is not included in its own rolling window. The supervised builder also creates anchor features from information available before the target time. Validation is chronological rather than random. These choices prevent future information from appearing in the inputs.

### Why fetch seven days for an hourly update?

The newest row needs enough history to calculate 24-hour lags and rolling features. Fetching only the last hour would produce incomplete features. The overlap is safe because the store uses idempotent upserts on `(city, timestamp)`.

## ## 7. Forecasting Design

At time `t`, the model predicts AQI at `t + h`, where `h` is one of 1, 2, 3, 6, 12, 24, 36, 48, 60 or 72 hours. The model receives:

- target-time weather/calendar/location features known or forecast for `t + h`;
- anchor pollution state observed at time `t`;
- the horizon `h` itself.

### Why make horizon a feature?

One model can learn how the relationship changes with lead time. This avoids ten separate models, simplifies registry and deployment, and allows the same global model to serve all cities and horizons.

### Why not recursively feed predictions back in?

Recursive forecasting compounds errors because each prediction becomes the next input. This design uses the observed anchor state plus forecast weather and treats horizon directly, avoiding a long chain of generated values.

### Candidate models

- Persistence baseline: future AQI equals current AQI.
- Ridge regression: regularized linear reference.
- RandomForest: nonlinear bagged-tree reference.
- XGBoost: gradient-boosted-tree candidate and production champion.

The baseline is important because a complex model should beat the simplest sensible forecast. The champion beats persistence overall and at horizons from +2 hours onward; at +1 hour persistence is better, which is reported honestly.

### Why use RMSE, MAE and R-squared?

- **RMSE**: penalizes large misses more strongly, which matters for hazardous spikes.
- **MAE**: is easy to interpret in AQI points.

- **R-squared** shows explained variance relative to a mean baseline.

### Results

Model	RMSE	MAE	R-squared
XGBoost	19.69	12.80	0.850
RandomForest	20.38	13.56	0.839
Ridge	22.85	16.01	0.797
Persistence	25.27	15.71	0.752

Error increases with horizon. RMSE is 6.10 at +1 hour and 27.55 at +72 hours; R-squared falls from 0.985 to 0.695. The five-fold walk-forward mean RMSE is 20.55 with standard deviation 3.53.

### Prediction intervals

The model is a point predictor. To communicate uncertainty, validation residuals are grouped by horizon and the 10th and 90th percentiles are added to each point prediction. This produces an approximately 80% horizon-aware interval. It is a practical empirical interval, not a guarantee of 80% coverage in every future regime.

## 8. SHAP and Statistical Analytics

### Global SHAP

Global SHAP ranks features by mean absolute contribution over many observations. The strongest drivers are current AQI, current PM2.5, the 24-hour AQI trend, temperature, time-of-day terms, pressure, humidity and wind-related features. The Analytics & SHAP dashboard displays the top 15 features.

### Per-city SHAP

For each city's peak forecast hour, TreeExplainer produces signed contributions. The dashboard shows a baseline AQI, the forecast result and the leading features that raise or lower the prediction. Positive and negative contributions are shown separately so the user can interpret direction, not just importance.

### Analytics dashboard

The Analytics view includes:

- global SHAP feature importance;
- historical AQI distribution histogram;
- AQI category breakdown;
- city-average ranking;
- monthly seasonal pattern;
- hourly diurnal pattern;
- Pearson correlation heatmap for AQI and weather variables;
- summary cards for observations, top driver, highest-risk city and strongest relationship.

The analytics client first tries the live FastAPI endpoints and falls back to the committed `statistics.json` and `shap\_global.json` snapshots when the backend is unavailable.

### Correlation is not causation

The correlation heatmap is descriptive. A strong relationship does not prove that changing a weather variable would cause AQI to change by the same amount. That is why the What-If feature is labeled a model simulation rather than causal inference.

## 9. Dashboard and API

### Five dashboard views

1. **Forecast:** city selector, current AQI meter, category/advice, 72-hour or daily chart, prediction band, city map, legend, alert and per-city explanation.
2. **Analytics & SHAP:** global model drivers and statistical charts.
3. **Model Evaluation:** all candidate models from each training run, champion history, per-horizon chart and walk-forward table.
4. **Monitoring:** PSI drift, forecast-error status and biggest misses.
5. **What-If:** live driver sliders and baseline-versus-scenario prediction.

### ### AQI meter and map

The current card uses a segmented meter rather than a bare number. The meter uses the EPA category colors and labels the current value and category. The city map uses public OpenStreetMap tiles and colored AQI markers; it does not require a map API key.

### ### Main API endpoints

Endpoint	Purpose
--- ---	
<code>`/api/health`</code>	Service health, forecast readiness, timestamp and artifact source
<code>`/api/predictions`</code>	Full 22-city forecast payload
<code>`/api/predictions/{city}`</code>	One city's forecast
<code>`/api/evaluation`</code>	Per-horizon and walk-forward metrics
<code>`/api/leaderboard`</code>	Champion/challenger history
<code>`/api/statistics`</code>	EDA distributions, correlations and patterns
<code>`/api/shap/global`</code>	Global SHAP feature importance
<code>`/api/explain/{city}`</code>	Detailed city explanation
<code>`/api/monitoring`</code>	Drift and realized forecast error
<code>`/api/predict`</code>	On-demand coordinate forecast
<code>`/api/whatif`</code>	Scenario simulation
<code>`/api/chat`</code>	Grounded LLM advisor

## ## 10. MLOps and Deployment

### ### Champion-challenger gate

Every training run records all candidates. The best candidate becomes the challenger. It is promoted only if its validation RMSE is strictly lower than the current champion's RMSE. Ties are rejected. The leaderboard records model name, version, metrics, timestamp, candidates and promotion result.

This prevents an unattended training run from replacing a good model merely because it is newer. It also creates an auditable model history visible in the dashboard.

### ### Hourly publication workflow

The hourly GitHub Actions workflow:

1. checks out the repository;
2. installs the project dependencies;
3. fetches recent data and updates the feature store;
4. runs batch inference with the existing champion;
5. commits and pushes ``data/processed/predictions.json``.

The daily workflow retrains, evaluates, monitors and commits the corresponding reports and model artifacts. Railway reads the repository snapshot with a short cache, while Vercel bundles static fallback copies.

### ### Why was the stale-data issue possible?

The original hourly workflow updated Hopsworks but did not run inference or commit a new forecast document. The website therefore continued showing the last inference snapshot. The workflow was corrected so feature refresh and forecast publication are part of the same hourly job.

### ### Deployment topology

- **\*\*GitHub Actions:\*\*** scheduled compute and artifact publication.
- **\*\*Hopworks:\*\*** managed feature store and model registry when credentials are present.
- **\*\*Railway:\*\*** Dockerized FastAPI backend.
- **\*\*Vercel:\*\*** static React/Vite frontend.
- **\*\*GitHub repository:\*\*** durable published JSON snapshots and source of truth.

### ### Static fallback

If `VITE\_API\_URL` is absent, the frontend reads `/data/\*.json` bundled in Vercel. The What-If and chat capabilities are hidden because they need a live backend. The analytics endpoints also have a live-to-static fallback so a temporary backend error does not make the analytics page unusable.

## ## 11. Monitoring, Alerts and Advisor

### ### Alerts

`check\_forecast()` scans the 72-hour curve, finds the peak and first threshold crossing, and assigns severity and health advice. The dashboard shows a banner when forecast severity is not `none`.

### ### PSI drift

Population Stability Index compares the recent feature distribution with a historical reference distribution. It is a monitoring signal, not a direct accuracy metric. Seasonal drift can be real and expected; the appropriate response is to inspect drift alongside realized forecast error rather than retrain blindly on every warning.

### ### Forecast error

Inference logs forecasts with timestamps and interval bounds. After the corresponding observations arrive, monitoring joins forecasts to actuals and reports errors and biggest misses. This creates a feedback loop for investigating model degradation.

### ### Grounded advisor and MCP

The advisor uses tool calls to read the actual forecast rather than inventing AQI values. The same domain tools are exposed through the Model Context Protocol. This separates language generation from authoritative data retrieval and gives the user plain-language guidance grounded in the project's own output.

## ## 12. Testing and Quality

The repository currently has **\*\*33 passing tests\*\*** covering AQI logic, feature engineering, supervised construction, statistics and storage reads. Important test themes include:

- AQI category and breakpoint behavior;
- per-city grouping and leakage prevention;
- chronological supervised data construction;
- statistics artifact structure;
- feature-store read behavior and fallback handling.

Additional validation performed for the submission:

- Vite production build passes;
- thesis PDF compiled successfully to 132 pages;
- live browser checks confirm the dashboard starts at `scrollY = 0`;
- Railway and Vercel report the same forecast timestamp;
- GitHub repository is public and includes the report PDF and EDA materials.

## ## 13. Interview Questions and Strong Answers

### ### Project and product



**\*\*Q:** Walk me through your project.\*\*

**\*\*A:\*\*** Start with the health problem, then explain the FTI architecture, global multi-horizon XGBoost model, champion gate, artifact publication and dashboard. End with the measured champion metrics and the explainability/monitoring layer.

**\*\*Q:** How is this different from a notebook?\*\*

**\*\*A:\*\*** It has scheduled ingestion, a feature-store boundary, a registry, model promotion governance, automated inference, monitoring, tests, a public API and a public dashboard. It continues working after the developer closes the laptop.

**\*\*Q:** Why React instead of Streamlit?\*\*

**\*\*A:\*\*** The final product needed a polished multi-view interface, interactive map, segmented AQI meter, responsive charts and clearer separation between frontend and API. Vite creates a static deployable bundle while FastAPI handles backend behavior.

**\*\*Q:** Why one global model instead of one model per city?\*\*

**\*\*A:\*\*** One global model shares statistical strength across cities, reduces artifact and maintenance count, supports new coordinates through location features, and keeps the registry simpler. The trade-off is that city-specific behavior may be less specialized.

### ### Data and modeling

**\*\*Q:** What is the most important data-science risk?\*\*

**\*\*A:\*\*** Temporal leakage. A random split or an unshifted rolling window can make the model appear excellent by allowing future information into training. I prevent it with shifted features, chronological splitting and walk-forward validation.

**\*\*Q:** Why is +1 hour worse than persistence?\*\*

**\*\*A:\*\*** Persistence is extremely strong at the shortest horizon because AQI is highly autocorrelated. The model improves over persistence from +2 hours onward. Reporting this exception is more credible than claiming a universal win.

**\*\*Q:** Why does error grow with horizon?\*\*

**\*\*A:\*\*** Weather forecast uncertainty grows, the current pollution anchor becomes less informative, and more unobserved events can occur. The metrics and intervals expose that degradation.

**\*\*Q:** Why XGBoost over LSTM?\*\*

**\*\*A:\*\*** The LSTM was evaluated fairly at +24 hours and slightly edged XGBoost for that single task, but XGBoost serves all ten horizons in one model, trains much faster, works naturally with tabular engineered features and has exact TreeSHAP explanations. The production choice is based on system-level trade-offs, not only one score.

**\*\*Q:** What does the model learn from the current AQI?\*\*

**\*\*A:\*\*** Current AQI and PM2.5 are strong SHAP drivers because pollution is persistent. That is useful signal, but it is why a persistence baseline is essential: the model must add value beyond simply carrying the current state forward.

**\*\*Q:** How would you improve the model?\*\*

**\*\*A:\*\*** Validate against ground stations, train with forecast-weather histories rather than observed future weather, calibrate intervals prospectively, add pollutant-level models, investigate satellite/fire signals, and let sustained drift/error trigger adaptive retraining.

### ### Explainability and evaluation

**\*\*Q:** What is a SHAP value?\*\*

**\*\*A:\*\*** It is a Shapley-based attribution of a prediction. Each signed value estimates how a feature moves the prediction relative to a baseline, and the contributions sum to the model output under the explainer's formulation.

**\*\*Q:** Global versus local SHAP?\*\*

**\*\*A:\*\*** Global importance summarizes average absolute impact across observations. Local



SHAP explains one city's one forecast, including direction: which features raise or lower that prediction.

**\*\*Q: Is correlation analysis causal?\*\***

**\*\*A:\*\*** No. The heatmap is descriptive. It helps understand data relationships and feature design, but it cannot establish that manipulating a correlated variable will change AQI.

**\*\*Q: How do you communicate uncertainty?\*\***

**\*\*A:\*\*** I group validation residuals by horizon and use the 10th and 90th percentiles as an 80% empirical interval. It communicates expected error growth but requires future coverage validation before being called a guaranteed confidence level.

**\*\*Q: How do you know the model is good?\*\***

**\*\*A:\*\*** It beats the persistence, Ridge and RandomForest candidates on the main chronological validation split, has R-squared 0.85, and is evaluated again with five walk-forward folds. I also publish per-horizon metrics instead of hiding lead-time degradation inside one average.

### ### Engineering and operations

**\*\*Q: What happens if Open-Meteo fails?\*\***

**\*\*A:\*\*** Requests retry transient 429/5xx errors with backoff. Each city is processed inside a failure boundary where possible, so one city does not necessarily discard the whole run. The Action status and logs make persistent failure visible.

**\*\*Q: What happens if the training job produces a worse model?\*\***

**\*\*A:\*\*** The champion-challenger gate rejects it and preserves the existing champion. The decision is written to the leaderboard.

**\*\*Q: How does the dashboard get fresh data?\*\***

**\*\*A:\*\*** The hourly workflow runs inference and commits `predictions.json`. Railway fetches the repository artifact at request time with a short cache. Vercel also has bundled snapshots for fallback.

**\*\*Q: Why use committed JSON artifacts?\*\***

**\*\*A:\*\*** They provide a simple, inspectable contract between scheduled batch compute and the serving layer. The backend does not need a live feature-store query for every page view, and the artifact history is auditable in Git.

**\*\*Q: What was the hardest production issue?\*\***

**\*\*A:\*\*** The stale-dashboard issue: the feature workflow succeeded but only updated the feature store; it did not publish a new inference artifact. The fix was to run inference and commit predictions in the hourly workflow, then verify timestamps across GitHub, Railway and Vercel.

**\*\*Q: How do you handle secrets?\*\***

**\*\*A:\*\*** Local values come from ignored environment files, while Actions uses GitHub Secrets and Railway uses deployment environment variables. No API key is required for Open-Meteo or the OpenStreetMap basemap.

### ### Behavioral answers

**\*\*Q: What did you learn?\*\***

**\*\*A:\*\*** A model metric is only one part of a reliable ML system. Data contracts, refresh semantics, observability, rollback behavior, artifact freshness and user trust matter just as much as model selection.

**\*\*Q: Tell me about a mistake.\*\***

**\*\*A:\*\*** I initially treated the hourly feature-store run as equivalent to a website refresh. It was not: features had changed but the served forecast document had not. Tracing the timestamp through the pipeline exposed the missing publication step. I fixed the workflow and added cross-service freshness checks.

**\*\*Q: What would you do in your first month at 10Pearls?\*\***

**\*\*A:\*\*** Understand the product and success metrics, reproduce the existing pipeline, add monitoring around the highest-risk data contract, make one small validated improvement, and document the result so the team can review or roll it back.

## ## 14. Rapid Review Before the Interview

Memorize these points:

1. **\*\*Problem:\*\*** current AQI is not enough; people need a 72-hour planning signal.
2. **\*\*Scope:\*\*** 22 cities, not Lahore only.
3. **\*\*Architecture:\*\*** hourly features and inference, daily training/evaluation, repository-backed JSON artifacts, FastAPI plus React.
4. **\*\*Model:\*\*** global XGBoost with horizon as a feature.
5. **\*\*Metrics:\*\*** RMSE 19.69, MAE 12.80, R-squared 0.85.
6. **\*\*Honesty:\*\*** persistence wins at +1 hour; the model wins from +2 hours onward.
7. **\*\*Leakage defense:\*\*** shifted rolling features and chronological validation.
8. **\*\*Uncertainty:\*\*** horizon-specific empirical residual intervals.
9. **\*\*Explainability:\*\*** global SHAP plus per-city signed contributions.
10. **\*\*Operations:\*\*** champion gate, hourly publication, daily retraining, PSI and realized-error monitoring.
11. **\*\*UI:\*\*** five views, AQI meter, OpenStreetMap markers, analytics heatmap and evaluation tables.
12. **\*\*Evidence:\*\*** 33 tests pass, live app is public, report and EDA are in GitHub.

## ## Appendix A: Useful Commands

```
```powershell
# Backend tests
pytest -q

# Build the frontend
Set-Location web
npm install
npm run build

# Run the backend locally from the repository root
$env:PYTHONPATH = "src"
uvicorn aqi.api.main:app --reload --port 8000

# Run the main pipelines
python -m aqi.pipelines.feature_pipeline
python -m aqi.pipelines.inference
python -m aqi.pipelines.training_pipeline
```
```

## ## Appendix B: Final Links

- GitHub: <https://github.com/Hasnain-irshad/10pearls-aqi-predictor>
- Dashboard: <https://www.10pearlsaqi.me>
- API health: <https://aqi-backend-production-5af4.up.railway.app/api/health>
- Report: `Thesis/Pearls\_AQI\_Predictor\_Project\_Report.pdf`